Owen Chilson

CSCI 430/530

Sarnath Ramnath

10/07/2026

Receive Payment from a Client

Business Process

The clerk requests to receive a payment from a client and provides the client's ID and the payment amount. The system searches for the client. If the client is found, the payment amount is credited to the client's account, reducing the amount the client owes.

If the payment is greater than the current balance, the remaining amount becomes a credit on the client's account. The warehouse system only records the account update; the actual financial transaction is handled separately.

For example, if the client has a $200.00 balance and pays $300.00, the client will have a $100.00 credit. If the client pays $100.00, the remaining balance will be $100.00.

Question 1: Queries Needed to Verify the Feature

Receiving a payment modifies the client's credit/debit balance. Therefore, a query that displays the client's account information and current balance can be used before and after the payment to verify that the feature was implemented correctly.

Data Modified	**Query Before Testing**	**Query After Testing**	**How This Verifies the Operation**
Client's credit/debit balance	Run Query Client Details and record the client's ID and current balance.	Run Query Client Details again and record the updated balance.	Verifies that the correct client's balance changed by exactly the payment amount. If the payment is greater than the amount owed, the resulting negative balance represents a credit on the client's account.

Example Tests

Suppose Client C1 has a balance of $200.00.

Test	**Before**	**Payment**	**Expected After**
Partial payment	$200.00	$100.00	$100.00 balance
Full payment	$200.00	$200.00	$0.00 balance
Overpayment	$200.00	$300.00	$100.00 credit

Question 2: Detailed Use Cases

Detailed Use Case for Receive Payment from a Client

Actor: Clerk (trusted)

Preconditions: The warehouse system is available to the clerk.

Postconditions: If the client is found and the payment amount is valid, the client's credit/debit balance is credited by the payment amount. If the payment exceeds the amount owed, the remaining amount becomes a credit. No actual financial transaction is performed by the warehouse system.

Actor	**System Response**
1. Request to receive a payment from a client.	2. Ask for the client ID.
3. Enter the client ID.	4. Search for the client. If the client does not exist, display "Client not found" and exit the operation.
	5. Display the client's current balance and ask for the payment amount.
6. Enter the payment amount.	7. Verify that the payment amount is greater than zero. If it is not, display an error and ask for the payment amount again.
	8. Credit the payment amount to the client's account.
	9. Obtain the client's updated balance.
	10. If the updated balance is positive, display the remaining balance. If it is zero, display a $0.00 balance. If it is negative, display the absolute value as a client credit.
	11. Display "Payment recorded" and exit the operation.

Detailed Use Case for Query Client Details

Actor: Clerk (trusted)

Actor	**System Response**
1. Request client details.	2. Ask for the client ID.
3. Enter the client ID.	4. Search for the client. If the client does not exist, display "Client not found" and exit the operation.
	5. Display the client's ID, name, address, and current credit/debit balance.

Question 3: Design a Solution

Receive Payment from a Client

The UserInterface handles interaction with the clerk. Warehouse acts as the facade and coordinates the operation. ClientList is responsible for finding the requested client, and Client is responsible for maintaining and modifying its own credit/debit balance.

Step	What the System Does	Responsible Class / Method	Design Choice
1	Receive the request to process a client's payment and obtain the client ID.	UserInterface	The UserInterface is responsible for interaction with the clerk.
2	Request the client from the warehouse system.	UserInterface → Warehouse.getClient(clientId)	The UI communicates through Warehouse instead of accessing ClientList directly.
3	Search for the requested client.	Warehouse → ClientList.search(clientId)	ClientList owns the collection of clients and is responsible for searches.
4	If the client does not exist, return a failure result.	Warehouse → UserInterface	No client data should be modified when an invalid client ID is supplied.
5	If the client exists, obtain and display the current balance and request the payment amount.	Client.getBalance() → Warehouse → UserInterface	Client owns its balance, while the UI is responsible for displaying it and requesting input.
6	Receive and validate the payment amount.	UserInterface	The UI ensures that the entered payment amount is greater than zero before requesting the account update.
7	Request that the payment be applied.	UserInterface → Warehouse.receivePayment(clientId, amount)	Warehouse acts as the facade and coordinates the business operation.
8	Credit the payment amount to the client's account.	Warehouse → Client.credit(amount)	Client owns the balance and is therefore responsible for modifying its own account information.
9	Obtain the updated balance.	Warehouse → Client.getBalance()	The updated balance is needed so the result can be displayed to the clerk.
10	Return the updated balance.	Client → Warehouse → UserInterface	Warehouse coordinates the operation while UserInterface handles presentation.
11	Display the remaining balance, $0.00 balance, or client credit.	UserInterface	The UI presents the result without directly changing any domain object.

Query Client Details

Step	What the System Does	Responsible Class / Method	Design Choice
1	Receive the request for client details and obtain the client ID.	UserInterface	The UI handles interaction with the clerk.
2	Request the client.	UserInterface → Warehouse.getClient(clientId)	The UI accesses the domain model through the Warehouse facade.
3	Search for the client.	Warehouse → ClientList.search(clientId)	ClientList owns and searches the client collection.
4	If the client does not exist, return a failure result.	Warehouse → UserInterface	The system reports an invalid client ID without modifying stored data.
5	Obtain the client's stored information and balance.	Client getters	The Client object is responsible for providing its own stored information.
6	Display the client's ID, name, address, and current balance.	UserInterface	This is a read-only query used to inspect the current client information.

Patterns Used

Facade: Warehouse acts as the facade between UserInterface and the domain classes. UserInterface does not directly search ClientList or modify the Client balance.

This keeps the user interface separate from the domain objects and gives Warehouse responsibility for coordinating the operation.

Question 4: Sequence Diagrams

Receive Payment from a Client

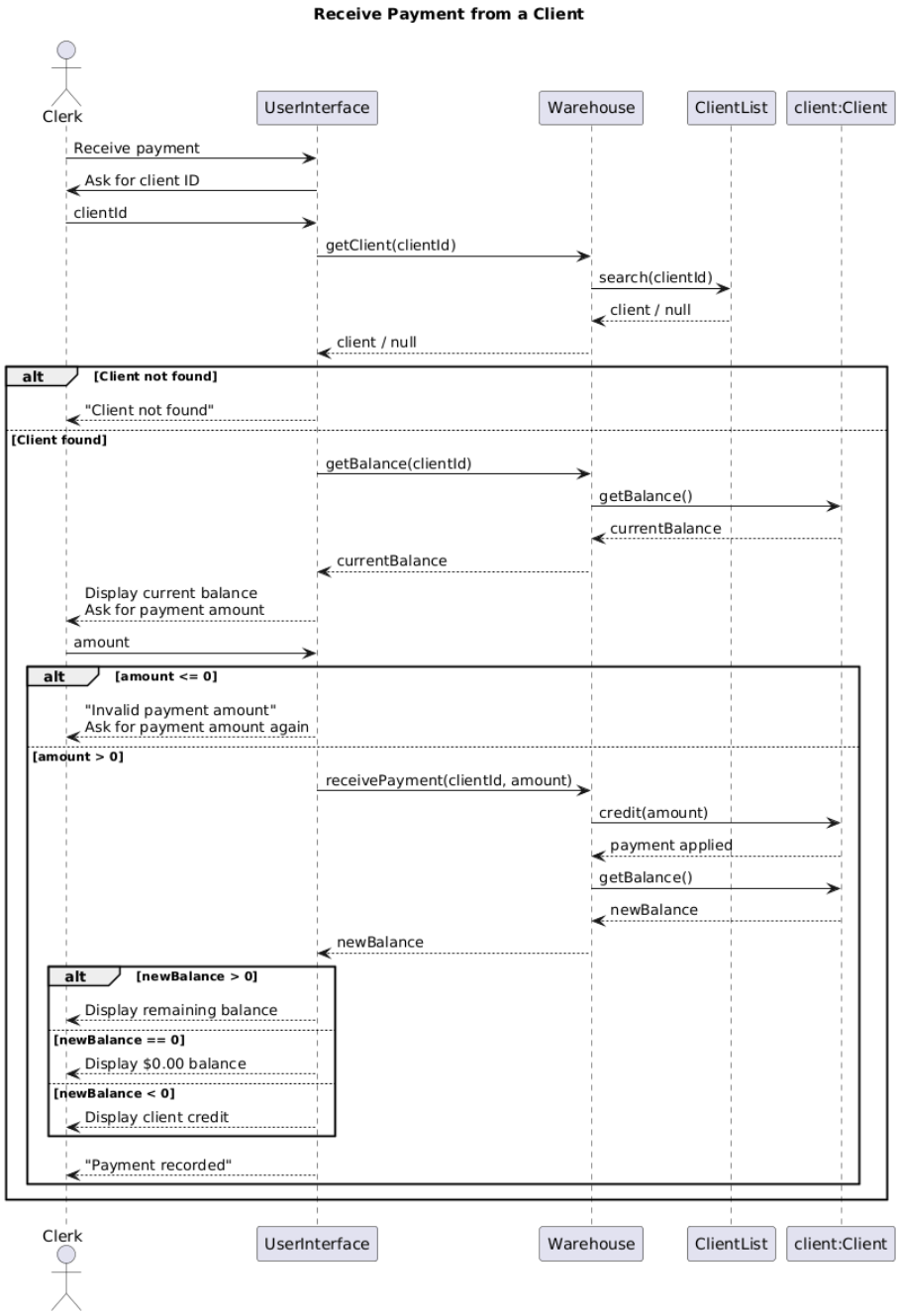


Figure 1: Receive Payment from a Client

Query Client Details

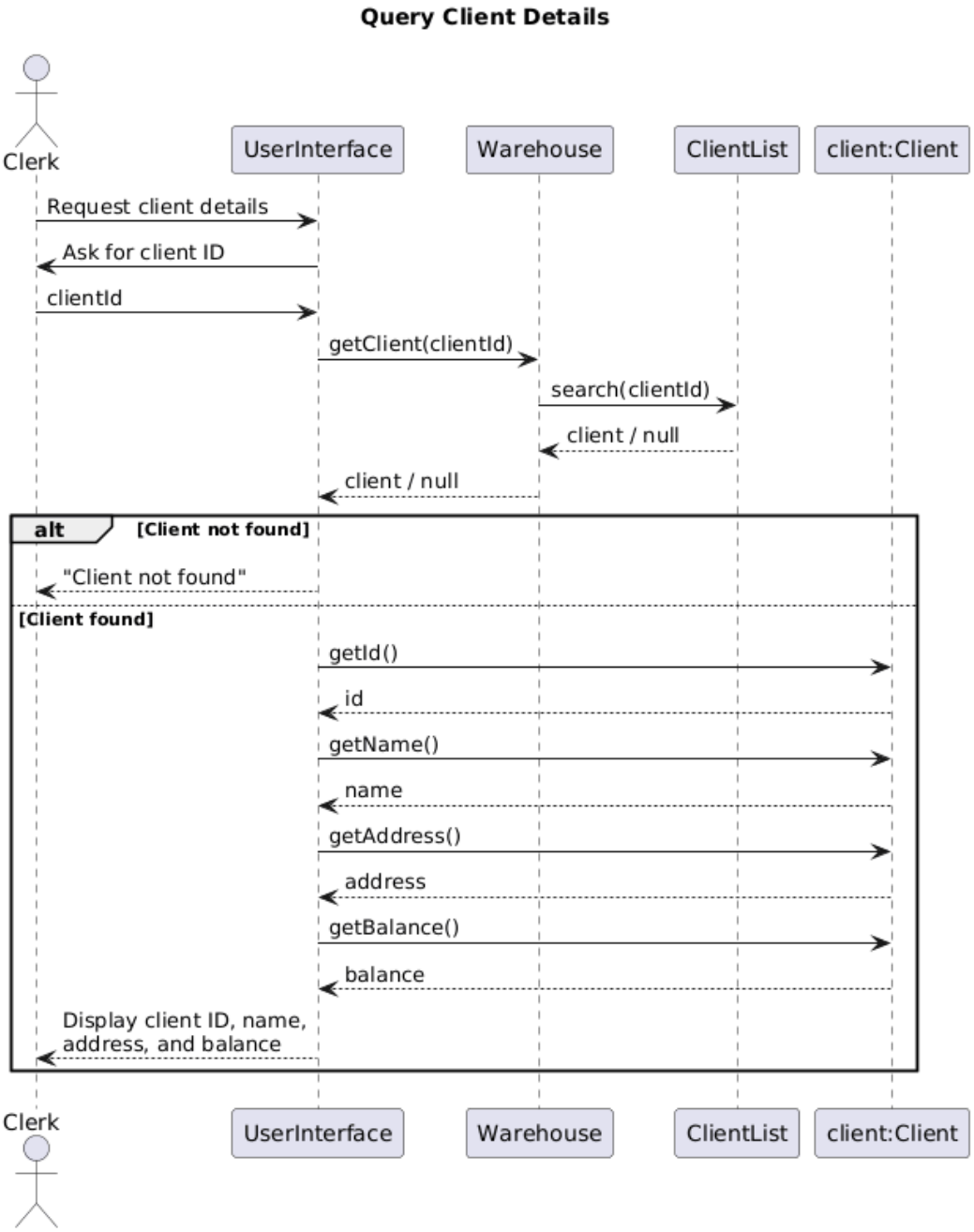


Figure 2: Query Client Details